

Chapter 15

AI Agents in LangChain

Understanding what AI agents are, why they exist, and building a ReAct-style agent from scratch using LangChain's `create_react_agent` and `AgentExecutor`.

WHAT THIS CHAPTER COVERS

- ▶ The real-world problem AI agents solve
- ▶ Formal definition and characteristics of an AI Agent
- ▶ LLM + Tools = Agent
- ▶ The ReAct design pattern (Reasoning + Acting)
- ▶ Agent vs. AgentExecutor
- ▶ Building a basic search-only ReAct agent
- ▶ Adding a custom weather API tool
- ▶ Why LangChain recommends LangGraph for production agents

AI Agents in LangChain

RECAP — CHAPTER 14

Chapter 14 covered Tool Calling: binding tools to an LLM with `.bind_tools()`, inspecting structured `tool_calls`, executing tools manually with `tool.invoke()`, and handling dependent arguments with `InjectedToolArg`. That entire workflow was **manually orchestrated** by the developer — every tool call, every argument injection, and every re-invocation of the LLM was written out step by step in code. This chapter introduces AI Agents, where that orchestration loop is handled automatically and autonomously by the framework itself.

1. What Problem Do AI Agents Solve?

A Relatable Scenario

Imagine planning a 7-day trip from Delhi to Goa on your own. The process typically breaks down into several independent, effortful stages:

1. **Booking travel** — visiting a site like MakeMyTrip or IRCTC, filling in a form (source, destination, date, class), comparing options, and booking a ticket — twice, for the onward and return journey.
2. **Booking accommodation** — searching hotels for the destination city, filtering by check-in/check-out dates, comparing price, reviews, and amenities, then paying.
3. **Planning activities** — researching top attractions, building a personalized day-by-day itinerary, and arranging local transport or entry tickets for each stop.

Each of these stages involves heavy research, comparison, and decision-making across multiple websites. For someone unfamiliar with navigating many different apps — for example, an elderly user — this entire process can take days and feels overwhelming.

How an AI Agent Changes the Experience

Now picture the same task handled through a chat interface backed by an AI agent. The user simply states a high-level goal in natural language:

```
"Can you create a budget travel itinerary from Delhi to Goa from 1st to 7th May?"
```

From this single instruction, the agent autonomously:

- Parses intent — source, destination, dates, duration, and the budget preference — and forms an internal goal such as *"plan affordable travel, stay, local movement, and activities over 7 days."*
- Searches train/flight APIs, compares price and duration, and proposes the cheapest and fastest option.
- Searches hotel options filtered by budget and rating, and proposes a stay.

- Recommends cost-effective local transport (for example, a scooter rental in Goa).
- Builds a day-wise activity itinerary from a knowledge base of popular attractions.
- Summarizes the full budget breakdown and, once approved, executes the actual bookings, sends invoices by email, and adds events and reminders to the user's calendar.

At every step the agent asks for confirmation, incorporates the user's preferences into memory, and adapts the plan if new information appears (for example, discovering a public holiday that closes certain attractions mid-trip).



Figure 15.1 — Manual, multi-website trip planning versus a single natural-language goal handed to an AI agent.

REAL WORLD INSIGHT

This same pattern generalizes beyond travel — customer support, education, healthcare scheduling, and countless other domains benefit from an autonomous agent that removes the burden of navigating many disconnected tools and interfaces.

🔄 Quick Revision — The Problem AI Agents Solve

- Multi-step real-world tasks (like trip planning) require visiting many different tools/websites and making many decisions manually.
- This manual process is slow, effortful, and inaccessible to less tech-savvy users.
- An AI agent lets a user state a high-level goal in natural language instead of specifying every step.
- The agent autonomously plans, compares options, executes actions, and adapts to new information.
- The same benefit applies to any domain with multi-step, tool-dependent workflows.

2. What Is an AI Agent?

Definition

IMPORTANT

An AI agent is an intelligent system that receives a high-level goal from a user and autonomously plans, decides, and executes a sequence of actions using external tools, APIs, and knowledge sources — all while maintaining context, reasoning over multiple steps, adapting to new information, and optimizing for the intended outcome.

In simpler terms: you give the agent a goal (for example, "*plan my Goa trip within budget*") without specifying exactly how to achieve it. The agent figures out the steps on its own, using whatever tools it has access to.

The Core Formula: LLM + Tools

Structurally, an AI agent is built from exactly two components:

- **An LLM**, used purely as the *reasoning engine*. Because an LLM understands natural language, it can interpret a task and decide how it should be broken down and executed.
- **A set of tools**, which give the agent the ability to actually take action — hitting APIs, querying databases, or performing a web search. An LLM by itself can reason and generate text, but it cannot fetch live data or perform real-world actions.

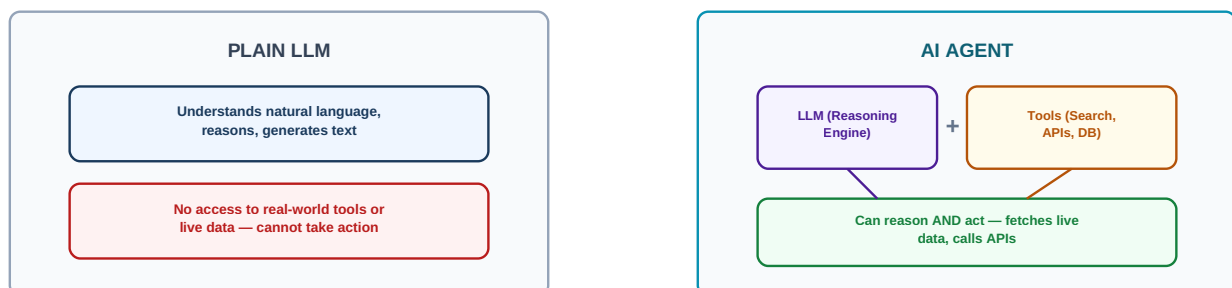


Figure 15.2 — A plain LLM can only reason and generate text; an AI agent combines an LLM with tools so it can also take action.

COMMON MISTAKE

Treating an LLM and an AI agent as the same thing. An LLM alone cannot book a ticket, hit a live weather API, or query a database — it can only reason over the text it is given. It becomes an agent only once tools are attached and it is given the autonomy to decide when to use them.

Quick Revision — What Is an AI Agent?

- An AI agent is an intelligent system that autonomously plans and executes a sequence of actions to achieve a high-level goal.
- It combines exactly two components: an LLM (reasoning engine) and a set of Tools (ability to act).
- The LLM decides what to do; the tools let it actually do it (search, hit APIs, query databases).
- A plain LLM cannot perform real-world actions — it can only reason and generate text.
- The user specifies the goal, not the exact steps; the agent figures out the steps itself.

3. Characteristics of an AI Agent

Characteristic	Description
Goal-driven	You provide the task; the agent figures out how to accomplish it — you do not need to specify the exact steps.
Plans autonomously	It can break a large problem down into smaller steps and execute them one by one.
Tool-aware	It knows which tools are available to it and, critically, when each tool should be used.
Maintains context	It keeps a running memory of everything done so far — the user, their preferences, and progress made.
Adaptive	If something changes mid-task (for example, an API stops working), it can re-plan and adjust its approach rather than failing outright.

INTERVIEW TIP

If asked to define an AI agent in an interview, lead with the two-part formula — **LLM as the reasoning engine + Tools for taking action** — and then support it with the five characteristics: goal-driven, autonomous planning, tool-awareness, context retention, and adaptiveness.

Quick Revision — Characteristics of an AI Agent

- Goal-driven: given a goal, not a procedure.
- Can independently plan and break down a problem into steps.
- Aware of its available tools and when to use each one.
- Maintains context/memory across the entire task.
- Adaptive: can re-plan when new information or failures occur.

4. ReAct: Reasoning + Acting

Definition

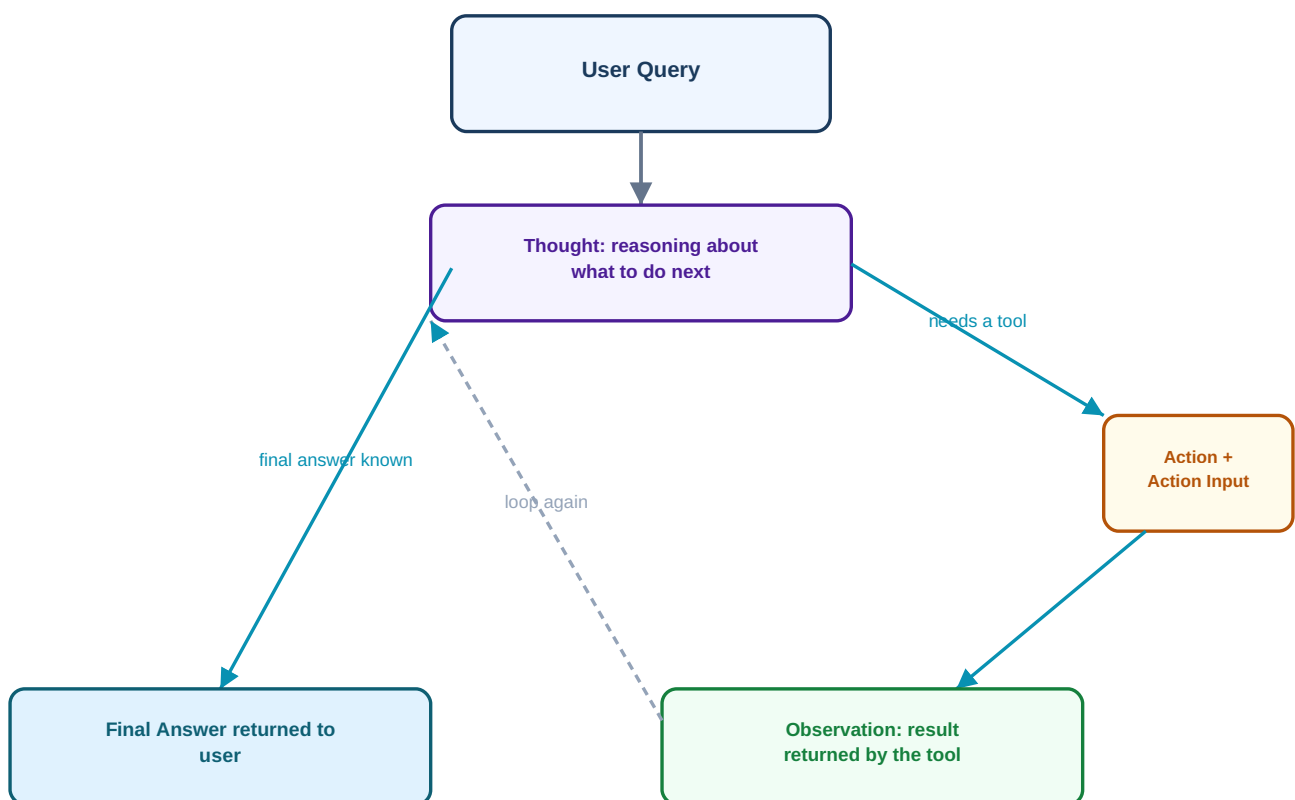
ReAct is a design pattern used in AI agents that stands for **Reasoning** + **Acting**. It allows a language model to interleave internal reasoning with external actions in a structured, multi-step process, rather than answering in a single turn like a typical LLM call.

Why It Exists

A normal LLM interaction is a single-turn exchange: the user sends a question, the model returns one response. ReAct breaks a problem into a repeating loop of three steps, executed as many times as needed:

1. **Thought** — the agent reasons about what to do next.
2. **Action** — if a tool is needed, the agent specifies which tool to call and with what input.
3. **Observation** — the result returned by the tool.

This Thought → Action → Observation loop repeats until the agent's Thought concludes *"I now know the final answer,"* at which point the loop breaks and the final answer is returned.



The Thought → Action → Observation cycle repeats until the agent decides it has the final answer.

Figure 15.3 — The ReAct loop: Thought leads to an Action, the Action produces an Observation, and the cycle repeats until a final answer is reached.

Worked Example

Query: "Can you tell me the population of the capital of France?"

Iteration	Thought	Action / Input	Observation
1	I first need to find the capital of France.	Search tool — "capital of France"	Paris
2	Now that I know the capital is Paris, I need its population.	Search tool — "population of Paris"	2.1 million
3	I now know the final answer.	— (no action)	—

Final Answer: "Paris is the capital of France and has a population of 2.1 million."

IMPORTANT

ReAct is particularly useful for **multi-step problems** (where one sub-answer is needed before the next step can proceed) and for tasks requiring tool access such as web search, database lookups, or API calls. Its biggest advantage is transparency — the entire reasoning trace is visible, so the user can always see what the agent is thinking and doing.

REAL WORLD INSIGHT

ReAct was introduced in the 2022 paper "ReAct: Synergizing Reasoning and Acting in Language Models." It remains one of the most influential and widely used agent design patterns; other patterns include Self-Ask with Search and various provider-specific agent architectures.

🔄 Quick Revision — ReAct

- ReAct = Reasoning + Acting, a design pattern for AI agents.
- It interleaves internal reasoning (Thought) with external actions (Action/Observation) in a loop.
- The loop runs: Thought → Action → Observation → repeat, until the agent knows the final answer.
- Best suited for multi-step problems and tasks that require tools (search, APIs, databases).
- Its reasoning trace is fully transparent and auditable to the user.
- Introduced in the 2022 paper 'ReAct: Synergizing Reasoning and Acting in Language Models.'

5. Agent vs. AgentExecutor

Building a ReAct agent in LangChain always involves two distinct objects, and understanding the division of labor between them is essential:

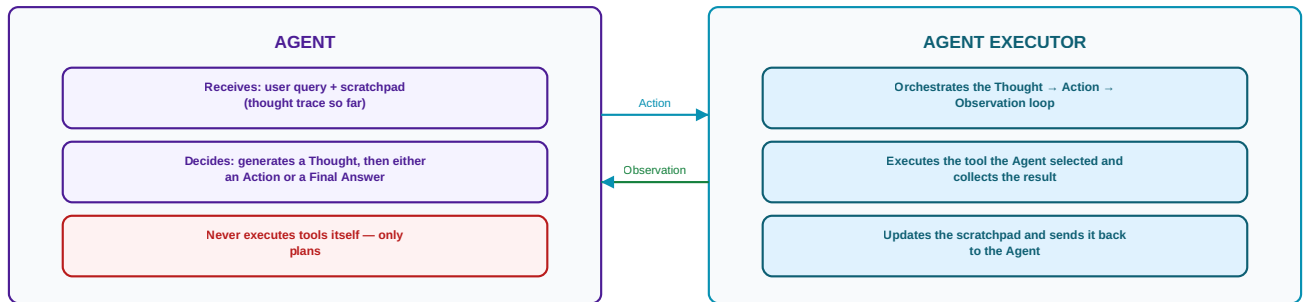


Figure 15.4 — The Agent decides what to do next; the AgentExecutor is responsible for actually running the loop and executing tools.

1. THE AGENT

The Agent is the "brain." Given the user's query and the current scratchpad (thought trace so far), it generates a Thought and then returns either an **AgentAction** (a tool to call, with input) or an **AgentFinish** (the final answer). It never executes anything itself.

2. THE AGENTEXECUTOR

The AgentExecutor is the "runtime." It orchestrates the entire Thought → Action → Observation loop: it sends the query and scratchpad to the Agent, executes whichever tool the Agent selected, collects the Observation, appends it to the scratchpad, and repeats — until the Agent returns an AgentFinish.

Object	Attributes	Meaning
AgentAction	tool, tool_input, log	Which tool to call, what input to give it, and the reasoning log so far.
AgentFinish	return_values, log	The final answer (in return_values) and the complete reasoning log.

COMMON MISTAKE

Confusing the Agent with the AgentExecutor. The Agent only thinks and decides — it never calls a tool directly. All actual tool execution, looping, and scratchpad management is the AgentExecutor's job.

Quick Revision — Agent vs. AgentExecutor

- The Agent is the reasoning brain: it produces a Thought and returns either an AgentAction or an AgentFinish.
- The AgentExecutor is the orchestrator: it runs the loop, executes tools, and maintains the scratchpad.
- AgentAction carries tool, tool_input, and log; AgentFinish carries return_values and log.
- The Agent never executes a tool itself — that responsibility belongs entirely to the AgentExecutor.
- The loop continues until the Agent returns AgentFinish.

6. Building a Basic ReAct Agent in LangChain

Goal of the First Version

The first agent built in this chapter is intentionally minimal: it is given exactly one tool — a DuckDuckGo web search tool. If the user's query needs information from the internet, the agent searches for it and answers based on what it finds.

STEP 1 — CREATE THE SEARCH TOOL AND THE LLM

```
from langchain_classic.agents import create_react_agent, AgentExecutor
from langsmith import Client
from langchain_groq import ChatGroq
from langchain_community.tools import DuckDuckGoSearchRun

# A ready-made web search tool -- no custom wrapping needed
search_tool = DuckDuckGoSearchRun()

# The LLM that will act as the agent's reasoning engine
llm = ChatGroq(model = "llama-3.3-70b-versatile")
```

STEP 2 — PULL THE REACT PROMPT FROM LANGCHAIN HUB

LangChain Hub is a community-hosted repository of prompts. Rather than writing a ReAct prompt from scratch, this chapter uses the well-tested `hwchase17/react` prompt (published by Harrison Chase, the creator of LangChain).

```

client = Client()

prompt = client.pull_prompt(
    "hwchase17/react",
    include_model = False,
    dangerously_pull_public_prompt = True
)

# Prompt skeleton (fetched from the hub):
# Answer the following questions as best you can. You have access to the following tools:
# {tools}
# Use the following format:
# Question: the input question you must answer
# Thought: you should always think about what to do
# Action: the action to take, should be one of [{tool_names}]
# Action Input: the input to the action
# Observation: the result of the action
# ... (this Thought/Action/Action Input/Observation can repeat N times)
# Thought: I now know the final answer
# Final Answer: the final answer to the original input question
# Begin!
# Question: {input}
# Thought: {agent_scratchpad}

```

Notice the placeholders: `{tools}` and `{tool_names}` are filled with the agent's available tools, `{input}` is the user's query, and `{agent_scratchpad}` is the running Thought/Action/Observation trace — empty at the start of the loop and growing with every iteration.

STEP 3 — CREATE THE AGENT AND THE AGENTEXECUTOR

```

# create_react_agent needs exactly 3 things: an LLM, a set of tools, and a prompt
agent = create_react_agent(
    llm = llm,
    tools = [search_tool],
    prompt = prompt
)

# AgentExecutor needs the agent and its tools -- it runs the actual loop
agent_executor = AgentExecutor(
    agent = agent,
    tools = [search_tool],
    verbose = True # prints the full Thought/Action/Observation trace
)

```

STEP 4 — INVOKE THE AGENT

```
response = agent_executor.invoke(  
    {"input": "How many metro lines are there in the capital of India?"}  
)  
print(response)
```

With `verbose=True`, running this query prints the full reasoning trace: the agent's Thought, its chosen Action (the search tool) and Action Input, the resulting Observation, and finally the Final Answer — exactly matching the ReAct loop described in Section 4.

BEST PRACTICE

Prefer a well-tested, community-maintained prompt (like `hwchase17/react` from LangChain Hub) over writing a ReAct prompt from scratch. These prompts have already been tuned to reliably produce the Thought/Action/Action Input/Observation format the parser expects.

Quick Revision — Building a Basic ReAct Agent

- A minimal ReAct agent needs just one tool (e.g. `DuckDuckGoSearchRun`) plus an LLM.
- LangChain Hub hosts reusable prompts; 'hwchase17/react' is the standard ReAct prompt.
- `create_react_agent(llm, tools, prompt)` builds the Agent object.
- `AgentExecutor(agent, tools, verbose=True)` builds the runtime that executes the loop.
- `agent_executor.invoke({'input': ...})` runs the query through the full ReAct loop.
- `verbose=True` prints the complete Thought / Action / Observation / Final Answer trace.

7. Improving the Agent: Adding a Custom Weather Tool

The agent is now extended with a second, custom tool: given a city name, it returns the current weather conditions (temperature, humidity, wind speed) using the WeatherStack API.

DEFINING THE CUSTOM TOOL

```
from langchain_core.tools import tool
import requests, os
from dotenv import load_dotenv

load_dotenv()

@tool
def get_weather_data(city: str) -> str:
    """This function fetches the current weather data for a given city."""
    api_key = os.getenv("WEATHER_STACK_API_KEY")
    url = f"https://api.weatherstack.com/current?access_key={api_key}&query={city}"
    response = requests.get(url)
    return response.json()
```

The `@tool` decorator turns this plain Python function into a LangChain tool automatically, using the function name, type hints, and docstring to build its schema and description.

COMMON MISTAKE

Never hardcode a real API key directly in shared code or notes. Load it from an environment variable (as shown above) and keep the actual key in a local `.env` file that is excluded from version control.

WIRING THE NEW TOOL INTO THE AGENT

Only one change is required compared to the basic agent: the new tool is added to the `tools` list passed to **both** `create_react_agent` and `AgentExecutor` — everything else (the LLM, the ReAct prompt) stays the same.

```
agent = create_react_agent(
    prompt = prompt,
    tools = [search_tool, get_weather_data],
    llm = llm
)

agent_executor = AgentExecutor(
    agent = agent,
    tools = [search_tool, get_weather_data],
    verbose = True
)

result = agent_executor.invoke(
    {"input": "What is the capital of Maharashtra. And also tell me the current temperature in that capital city."}
)
print(result)
```

TRACE WALKTHROUGH

For a query like *"Find the capital of Madhya Pradesh, then find its current weather condition,"* the resulting ReAct trace looks like this:

Iteration	Thought	Action	Observation
1	I should first find out the capital of Madhya Pradesh.	DuckDuckGo search — "capital of Madhya Pradesh"	Bhopal is the capital.
2	Now that I know the capital is Bhopal, I can use <code>get_weather_data</code> to check its current weather condition.	<code>get_weather_data("Bhopal")</code>	Partly cloudy, 40°C
3	I now know the final answer.	—	—

Final Answer: *"The capital of Madhya Pradesh is Bhopal and the current weather condition is partly cloudy with a temperature of 40°C."*

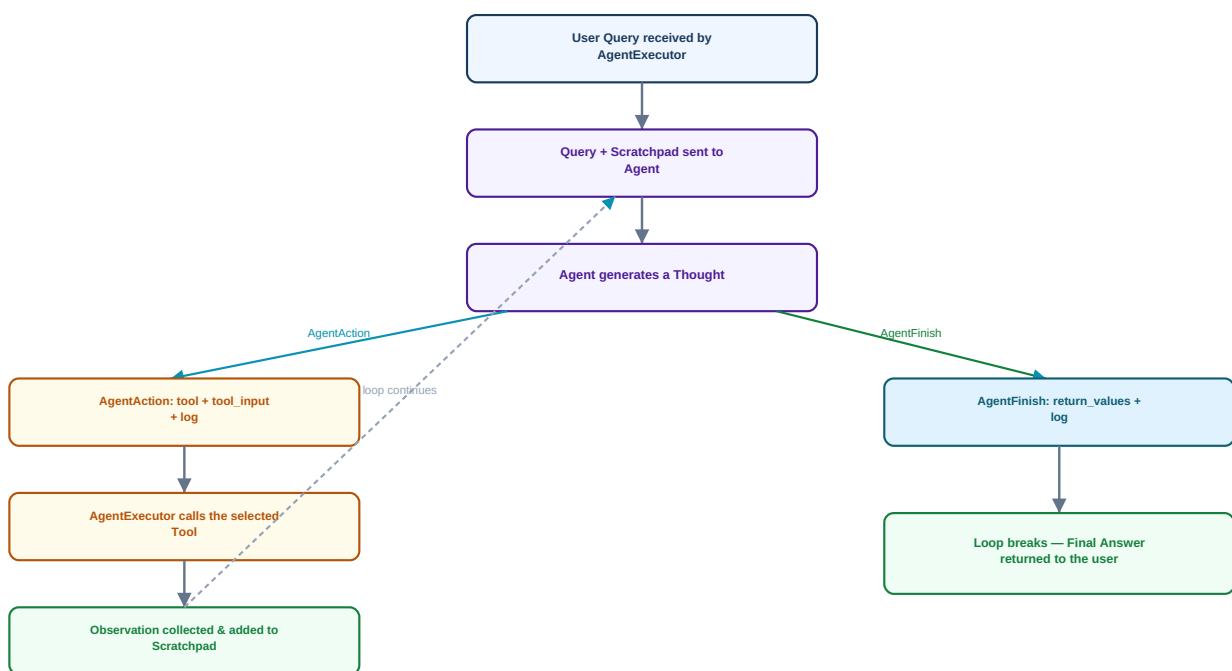


Figure 15.5 — The complete AgentExecutor ↔ Agent loop: every Thought branches into either an AgentAction (continue the loop) or an AgentFinish (return the final answer).

INTERVIEW TIP

Be ready to explain why the tools list appears twice — once inside `create_react_agent` (so the prompt can describe the available tools to the LLM) and once inside `AgentExecutor` (so the executor knows which tool objects to actually call at runtime).

Quick Revision — Adding a Custom Tool

- A custom tool is created with the `@tool` decorator around a typed, documented Python function.
- The new tool must be added to the tools list passed to both `create_react_agent` and `AgentExecutor`.
- The LLM, prompt, and `AgentExecutor` invocation pattern remain unchanged when adding a tool.
- The agent autonomously decides, at each Thought step, which of its available tools to use.
- Sensitive values like API keys should always be loaded from environment variables, never hardcoded.

8. Is LangChain Still Recommended for Production Agents?

The `create_react_agent` + `AgentExecutor` approach shown in this chapter works and is useful for understanding how agents function internally. However, it is no longer LangChain's recommended approach for building production-grade, scalable AI agents.

IMPORTANT

LangChain's own documentation now recommends **LangGraph** — a separate, purpose-built library from the LangChain team — for building robust, scalable AI agents. The classic `create_react_agent/AgentExecutor` pattern remains functional and is valuable for learning the underlying mechanics, but is considered outdated for serious production use.

This chapter deliberately used the classic approach for two reasons: first, to demonstrate that building an agent with LangChain directly is entirely possible, and second, to give a clear, transparent picture of how an agent's internal Thought/Action/Observation loop actually works — understanding gained here carries forward directly into learning LangGraph.

REAL WORLD INSIGHT

Understanding this chapter deeply means the ReAct loop, Agent, and `AgentExecutor` could, in principle, be implemented entirely from scratch in plain Python without relying on `create_react_agent` or the `AgentExecutor` class at all — a useful exercise for solidifying the underlying mechanics.

Quick Revision — LangChain vs. LangGraph for Agents

- `create_react_agent` + `AgentExecutor` is functional but considered outdated for production-scale agents.
- LangChain's own documentation recommends LangGraph for building robust, scalable AI agents.
- This chapter's classic approach was taught specifically to build intuition for the ReAct loop's internals.
- That underlying intuition (Thought/Action/Observation, Agent vs. `AgentExecutor`) transfers directly to LangGraph.

K. Key Takeaways

- **The Problem:** Multi-step, multi-tool real-world tasks (like trip planning) are effortful to execute manually; AI agents let a user state a high-level goal instead of every step.
- **Definition:** An AI agent autonomously plans, decides, and executes a sequence of actions using external tools to achieve a user-given goal, while maintaining context and adapting to new information.
- **Core Formula:** Agent = LLM (reasoning engine) + Tools (ability to act). A plain LLM alone cannot take real-world action.
- **Characteristics:** Goal-driven, capable of autonomous planning, tool-aware, context-maintaining, and adaptive.
- **ReAct:** A design pattern (Reasoning + Acting) that repeats a Thought → Action → Observation loop until the agent reaches a final answer.
- **Agent vs. AgentExecutor:** The Agent only thinks and decides (AgentAction or AgentFinish); the AgentExecutor orchestrates the loop and actually executes tools.
- **Building an Agent:** `create_react_agent(llm, tools, prompt)` builds the Agent; `AgentExecutor(agent, tools)` builds the runtime; `agent_executor.invoke()` runs it.
- **LangChain Hub:** A community repository of reusable prompts, such as the standard 'hwchase17/react' ReAct prompt.
- **Custom Tools:** Built with the `@tool` decorator and added to the tools list in both `create_react_agent` and `AgentExecutor`.
- **Production Note:** LangChain now recommends LangGraph, not `create_react_agent/AgentExecutor`, for building scalable, production-grade agents.

R. Revision Sheet

AI Agent

An intelligent system that autonomously plans and executes actions to achieve a user-given high-level goal, using tools and maintaining context across multiple steps.

Agent = LLM + Tools

The LLM provides reasoning; tools provide the ability to act (search, hit APIs, query databases). Neither alone makes an agent.

ReAct

Reasoning + Acting: a design pattern where the model loops through Thought, Action, and Observation steps until it reaches a final answer.

Agent

The "brain" — given a query and scratchpad, generates a Thought and returns either an AgentAction or an AgentFinish. Never executes tools itself.

AgentExecutor

The "runtime" — orchestrates the Thought/Action/Observation loop, executes tools, and updates the scratchpad on every iteration.

AgentAction / AgentFinish

AgentAction carries (tool, tool_input, log); AgentFinish carries (return_values, log) and signals the loop should stop.

create_react_agent()

Builds a ReAct Agent from an LLM, a list of tools, and a prompt (commonly pulled from LangChain Hub, e.g. `hwchase17/react`).

LangChain Hub

A community-hosted repository of reusable, pre-tested prompts for building agents and other LangChain components.

Agent Scratchpad

The running Thought/Action/Observation trace for the current query; starts empty and grows with each loop iteration.

LangGraph

LangChain's recommended library for building scalable, production-grade AI agents, replacing the classic `create_react_agent/AgentExecutor` pattern.

I. Interview Questions

Q1. What is an AI agent, and how does it differ from a plain LLM call?

Q2. What are the two core components that make up every AI agent?

Q3. List and briefly explain the key characteristics of an AI agent.

Q4. What does ReAct stand for, and what problem does it solve compared to a single-turn LLM interaction?

Q5. Walk through the Thought → Action → Observation loop with an example of your choice.

Q6. What is the difference between an Agent and an AgentExecutor in LangChain?

Q7. What are the three arguments required by `create_react_agent()`, and what does each one do?

Q8. What are AgentAction and AgentFinish, and what attributes does each carry?

Q9. Why must the tools list be passed to both `create_react_agent` and `AgentExecutor` separately?

Q10. Why does LangChain now recommend LangGraph over `create_react_agent`/`AgentExecutor` for production agents?

F. Further Reading

- [LangChain documentation](#) — Agents concepts and API reference
- [LangChain Hub](#) — community-hosted prompt repository (hwchase17/react and others)
- Paper: "ReAct: Synergizing Reasoning and Acting in Language Models" (Yao et al., 2022)
- [LangGraph documentation](#) — LangChain's recommended framework for production-grade, scalable AI agents